



آزمایشگاه معماری کامپیوتر

نیم سال سوم ۱۴۰۴-۰۵

استاد: دکتر حمید سربازی آزاد

دانشکده مهندسی کامپیوتر

معین آعلی - ثمین اکبری

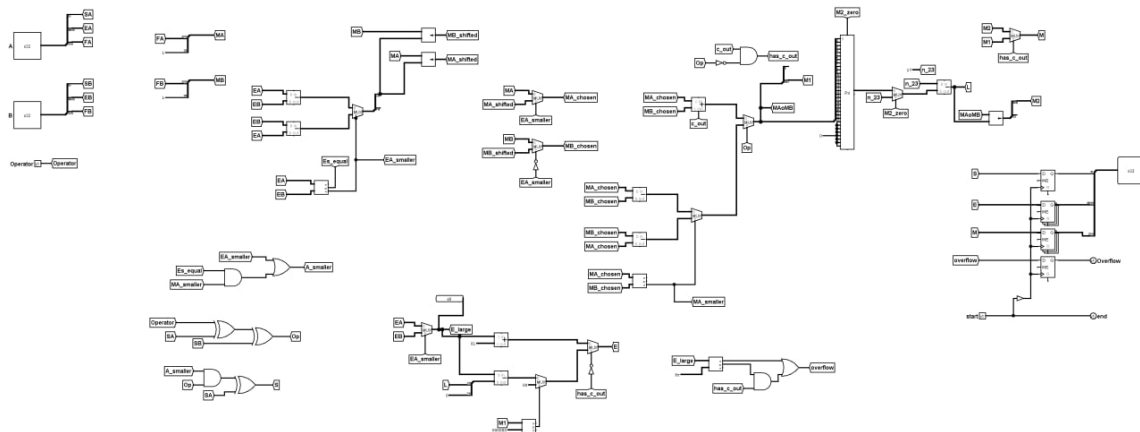
جمع کننده/تفریق کننده ممیز شناور

آزمایش چهارم

خلاصه

در این آزمایش یک جمع کننده/تفریق کننده ممیز شناور ۳۲ بیتی مطابق استاندارد IEEE754 در نرم افزار Logisim طراحی و پیاده سازی شده است. مدار به صورت تک سطحی و ترکیبی از قطعات آماده کتابخانه ی Arithmetic و Plexers ساخته شده: Comparator برای مقایسه ی نماها، Shifter برای هم تراز سازی و نرمال سازی مانتیس، Subtractor/Adder برای ترکیب مانتیس ها، و Priority Encoder برای یافتن موقعیت اولین بیت ۱ در مرحله ی نرمال سازی.

دیاگرام کلی مدار



هدف آزمایش

هدف از این آزمایش، آشنایی عملی با قالب نمایش IEEE754 و پیاده سازی یک واحد جمع/تفریق ممیز شناور ۳۲ بیتی است. برخلاف جمع صحیح، جمع دو عدد ممیز شناور به مراحل هم تراز سازی نما، جمع/تفریق مانتیس، و نرمال سازی نتیجه نیاز دارد. اهداف مشخص این آزمایش عبارت اند از:

- آشنایی عملی با قالب نمایش IEEE754 و نحوه ی رمزگشایی/رمزگذاری آن با علامت، نما و مانتیس.
- پیاده سازی مرحله ی هم تراز سازی نما (Alignment) پیش از جمع دو عدد ممیز شناور.
- پیاده سازی جمع/تفریق مانتیس با در نظر گرفتن علامت عملوندها و رقم نقلی.
- پیاده سازی مرحله ی نرمال سازی نتیجه و تشخیص سرریز نما.

قالب IEEE754

هر عدد ۳۲ بیتی به صورت زیر رمزگذاری می شود:

$$\text{Value} = (-1)^S \times 1.M \times 2^{E-127}$$

توضیح	فیلد	بیت‌ها	پهنای
0 مثبت، 1 منفی	S (علامت)	31	1
نمای بایاس شده با بایاس 127	E (نما)	23..30	8
قسمت کسری؛ بیت پنهان 1. صریح ذخیره نمی‌شود	M (مانتیس)	0..22	23

رابط مدار

توضیح	سیگنال	جهت	پهنای
عملوندها، قالب IEEE754	B, A	ورودی	32
0 جمع ($A + B$)، 1 تفریق ($A - B$)	Operator	ورودی	1
حاصل جمع یا تفریق، قالب IEEE754	C	خروجی	32
سرریز نما (قفلی شده)	Overflow	خروجی	1
با لبه‌ی بالارونده، محاسبه‌ی جاری را روی خروجی قفل می‌کند	start	ورودی	1
هم‌زمان با start فعال است؛ اعتبار خروجی را نشان می‌دهد	end	خروجی	1

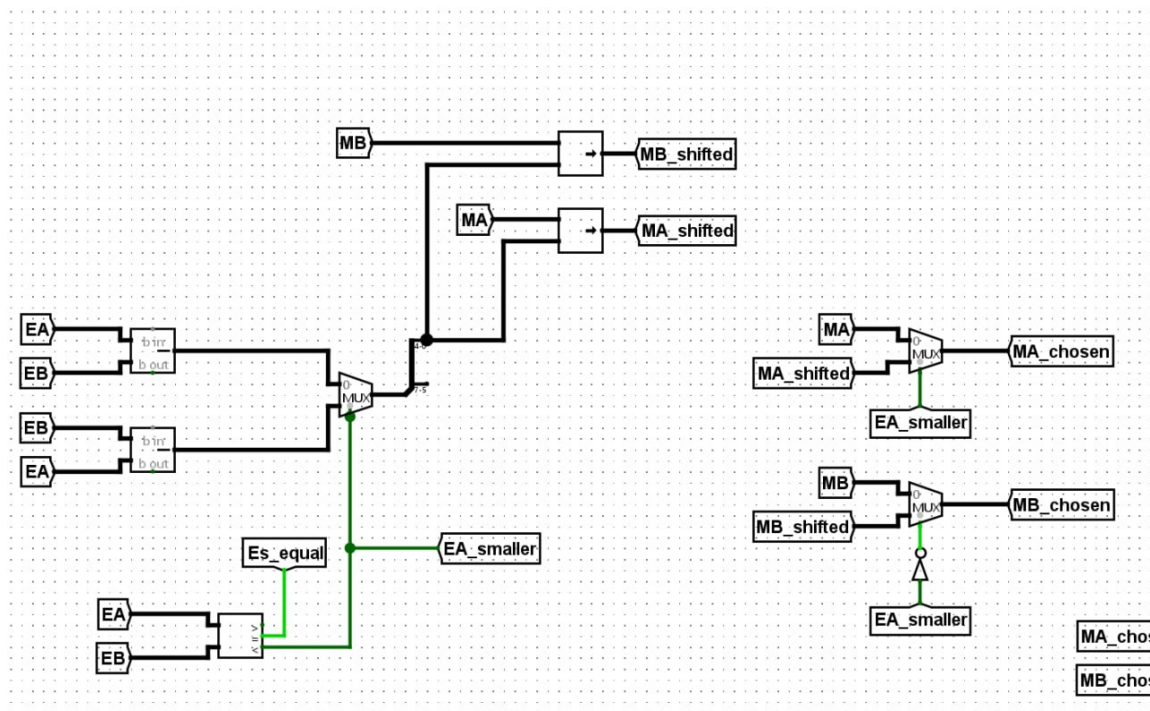
مدار در پنج مرحله کار می‌کند که در ادامه توضیح داده می‌شوند.

مرحله ۱ — استخراج فیلدها و هم‌ترازسازی نما

با یک Splitter روی هرکدام از A و B، فیلدهای S، E و M جدا می‌شوند و بیت پنهان 1 به ابتدای مانتیس اضافه می‌شود تا یک عدد 24 بیتی M ساخته شود (MA و MB).

نماها با یک Comparator (حالت unsigned) مقایسه می‌شوند تا $E_{\text{large}} = \max(E_A, E_B)$ مشخص شود. اختلاف نماها با یک Subtractor محاسبه و مانتیس عملوند کوچک‌تر با یک Shifter (شیفت منطقی راست) به همان اندازه شیفت داده می‌شود تا هر دو مانتیس روی مقیاس E_{large} هم‌تراز شوند:

$$M_{\text{small,aligned}} = M_{\text{small}} \gg (E_{\text{large}} - E_{\text{small}})$$



مرحله ۲ — تعیین عملیات مؤثر و علامت

چون جمع دو عدد با علامت مخالف معادل تفریق است (و برعکس)، عملیات مؤثر روی مانتیس‌ها با XOR کردن Operator و علامت عملوندها به دست می‌آید:

$$Op = Operator \oplus S_B$$

هم‌زمان اندازه‌ی دو عملوند به‌طور کامل (نه فقط نما) با ترکیب خروجی Comparator نماها و یک Comparator روی مانتیس‌ها مقایسه می‌شود:

$$A_{\text{smaller}} = (E_A < E_B) \vee ((E_A = E_B) \wedge (M_A < M_B))$$

علامت نتیجه از عملوند بزرگ‌تر گرفته می‌شود؛ در حالت $Op = 1$ (تفریق مؤثر) اگر عملوند کوچک‌تر انتخاب‌شونده تغییر کند، علامت نیز معکوس می‌شود.

مرحله ۳ — جمع/تفریق مانتیس

بسته به Op ، مانتیس‌های هم‌تراز شده وارد یک Adder ۲۴ بیتی یا یک Subtractor ۲۴ بیتی می‌شوند (تفریق همیشه به‌صورت بزرگ‌تر منهای کوچک‌تر انجام می‌شود، پس هرگز رقم قرضی ایجاد نمی‌کند). در حالت جمع، بیت رقم نقلی خروجی (has_c_out) نشان می‌دهد نتیجه از ۲۳ بیت (یعنی از بازه‌ی 1.xxx) فراتر رفته و باید یک بیت به راست شیفت داده و نمای خروجی یک واحد افزایش یابد. در این حالت یک رقم ۱ داریم و نیازی به شیفت بیشتر عدد برای نرمال‌سازی نیست.

مرحله ۴ — نرمال‌سازی و تشخیص سرریز

اگر جمع رقم نقلی تولید نکرده باشد (یا عملیات تفریق بوده)، ممکن است بیت پیشرو (1) در موقعیتی پایین‌تر از بیت ۲۳ افتاده باشد. یک Priority Encoder موقعیت اولین بیت ۱ مانتیس ۲۴ بیتی را پیدا می‌کند؛ تعداد شیفت لازم برای نرمال‌سازی از تفاضل این موقعیت با ۲۳ به دست می‌آید و مانتیس با یک Shifter نهایی به همان اندازه به

سرریز نما با مقایسه‌ی نمای نهایی (unsigned) با مقدار 0xFE (۲۵۴) تشخیص داده می‌شود — بزرگ‌ترین نمای مجاز پیش از رزروشدن کد 0xFF:

مرحله ۵ — واحد قفل خروجی (end/start)

بخش‌های ۱ تا ۴ کاملاً ترکیبی‌اند و همیشه در حال محاسبه‌ی مقدار جدید بر اساس A، B و Operator جاری هستند. برای اینکه خروجی فقط با فرمان صریح به‌روز شود، هرکدام از چهار فیلد نتیجه (Sign، Exponent، Mantissa، Overflow) از یک Register عبور می‌کند پیش از رسیدن به پایه‌ی خروجی:

$$Q \leftarrow D \quad \text{در لبه‌ی بالارونده‌ی } clk$$

هر چهار رجیستر با یک کلاک مشترک، از start ساخته می‌شوند. تا وقتی $start = 0$ است، رجیسترها آخرین مقدار قفل‌شده را نگه می‌دارند؛ با لبه‌ی بالارونده‌ی start، مقدار لحظه‌ای محاسبه‌شده ثبت و روی خروجی ظاهر می‌شود. سیگنال end مستقیماً از همان پایه‌ی start گرفته می‌شود (از طریق یک گیت Buffer که آن را از کلاک جدا نگه می‌دارد) و هم‌زمان با start فعال است — چون منطق محاسبه ترکیبی است، به محض عبور لبه‌ی کلاک مقدار جدید در دسترس است و نیازی به تأخیر یا شمارش سیکل نیست.

